

WEEK 01

Threads & Concurrency Basics

Theory Pack

All the concepts for the week, in one place.

*Threads will race.
Memory will be corrupted.
Programs will misbehave in confusing ways.*

Then we will fix all of it.

INSIDE

- 01** Processes vs Threads
- 02** Shared Memory: What Threads Actually Share
- 03** Race Conditions: The Mayhem Begins
- 04** Pthreads Basics

SECTION 01

Processes vs Threads

Before we can write a multithreaded program, we have to be clear about what a thread is. This section builds the concept up from scratch, starting with what a program even is in the first place.

What is a program?

A program, as a file on your disk, is just bytes. When you compile `hello.c` with `gcc hello.c -o hello`, you get a file called `hello` that contains the compiled machine instructions, some constant data, and some metadata about where everything goes. That file is dead. It does nothing on its own. It is a recipe.

A program *running* is something different. When you type `./hello` in your terminal, the operating system reads the file, sets up memory, loads the instructions, and starts the CPU executing them. That live, running instance of the program is called a **process**.

A program is to a process what a recipe is to a meal being cooked. The recipe sits in the cookbook. The meal exists only while someone is making it.

What is a process?

A process is an instance of a running program, along with everything the operating system needs to keep it alive: the machine code loaded into memory, a memory area for its variables and call stack, a set of CPU register values (the current state of the CPU as far as this program is concerned), a set of open files and other resources the OS is managing on its behalf, and bookkeeping like its process ID (pid), parent process, and permissions.

Crucially: **each process has its own private memory**. If you run `./hello` twice, you get two processes, each with its own separate copy of the program's memory. Neither can see the other's variables. They are isolated by the operating system. This isolation is what stops your browser from being able to read your password manager's memory, even though they're running on the same machine.

You can see processes on your system right now. On Linux, run:

```
$ ps aux
```

Every line is a process. Your shell, your editor, your terminal — all separate processes, all isolated from each other.

So what is a thread?

Processes are isolated, which is great for safety — but it means two processes can't easily cooperate on the same data. And there's a lot of overhead to creating one.

Sometimes a single program *wants* to do many things at once. A web browser wants to render the page while downloading the next image while playing a video. A game wants to simulate physics, handle player input, and stream the next level's assets simultaneously. Forcing each of these into a separate process is

awkward — they'd have to communicate through pipes or shared memory regions, and creating processes is comparatively expensive.

So operating systems support a lighter-weight idea. Inside a single process, you can have **multiple threads of execution** — multiple, independently-scheduled streams of CPU instructions, all sharing the same memory. A thread has its own program counter (where it is in the code), its own stack (its own local variables and function call history), and its own register values. But all the threads in a process share the same code, the same heap, the same global variables, and the same open files and resources.

If a process is a house, threads are the people inside it. The people each have their own activity, their own movement, their own thoughts — but they all share the same kitchen, the same furniture, the same WiFi.

This is what makes threads powerful: they can cooperate on shared data with essentially zero overhead. Two threads can both increment the same counter just by writing `counter++`.

And this is what makes threads dangerous: they can cooperate on shared data with essentially zero overhead. When two threads both write `counter++` at the same time, the operating system doesn't stop them. The hardware doesn't stop them. They just do it. And if you didn't write your code very carefully, the result is wrong.

That, in one paragraph, is the entire reason this project exists.

Side-by-side comparison

	Process	Thread
Memory	Private	Shared with siblings
Creation cost	Heavy (new address space)	Light (just a stack and bookkeeping)
Sibling communication	Hard — pipes, sockets, IPC	Trivial — shared variables
Crash safety	Isolated	Not isolated — one thread crashing usually kills the whole process
Scheduling unit	Yes	Yes — modern OSes schedule threads, not processes

That last row matters. The OS scheduler thinks in terms of threads. A program with one thread is the special case; a program with multiple threads gives the OS more independent things to schedule. On a machine with 8 cores, a program with 8 threads can actually run 8 things at once. A program with 1 thread will only ever use 1 core, no matter how good your hardware is.

Concurrency vs Parallelism

Concurrency is *dealing with* multiple things at once. The threads exist, they take turns, the program structure handles multiple things in flight. You can have concurrency on a single-core CPU — the OS just rapidly switches between threads.

Parallelism is *doing* multiple things at once. Genuinely, physically, at the same instant. This requires multiple CPU cores.

Most race conditions can appear with just concurrency — even on a single core. The OS can pause your thread at the worst possible moment between two instructions. The bugs do not need multiple cores to show up. They just become more frequent with parallelism.

SECTION 02

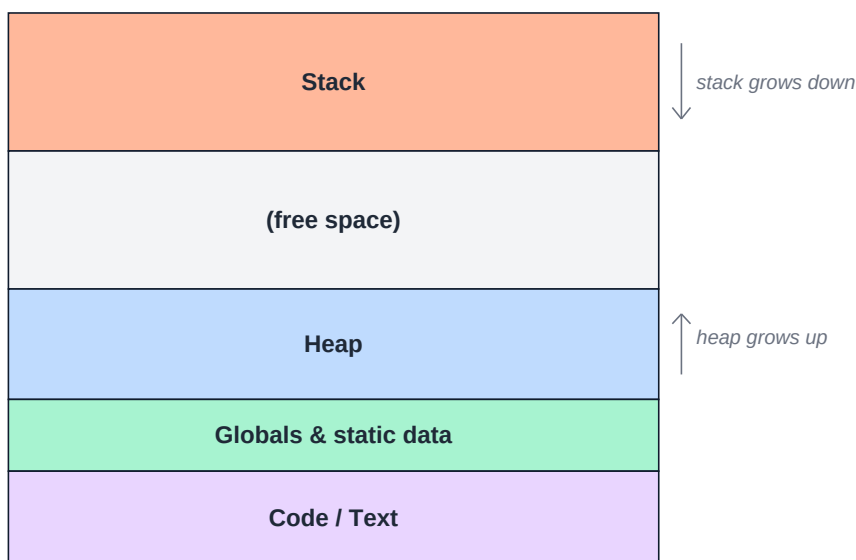
Shared Memory: What Threads Actually Share

Threads share memory. That's true but vague. To write correct multithreaded code — and to recognize the bugs you're about to create — you need to know exactly which parts of a program's memory are shared between threads, and which aren't.

Memory layout of a process

When a program is loaded into memory and starts running, the operating system arranges its memory into roughly four regions. The exact layout depends on the OS and architecture, but the idea is universal.

high addresses



low addresses

Process memory layout. A single thread uses the single stack at the top.

The four regions in detail

Code (text): The compiled machine instructions. Read-only — your program can't accidentally overwrite its own code.

Globals and static data: Global variables and static variables. Allocated when the program starts, exist for the entire lifetime of the program, at fixed addresses.

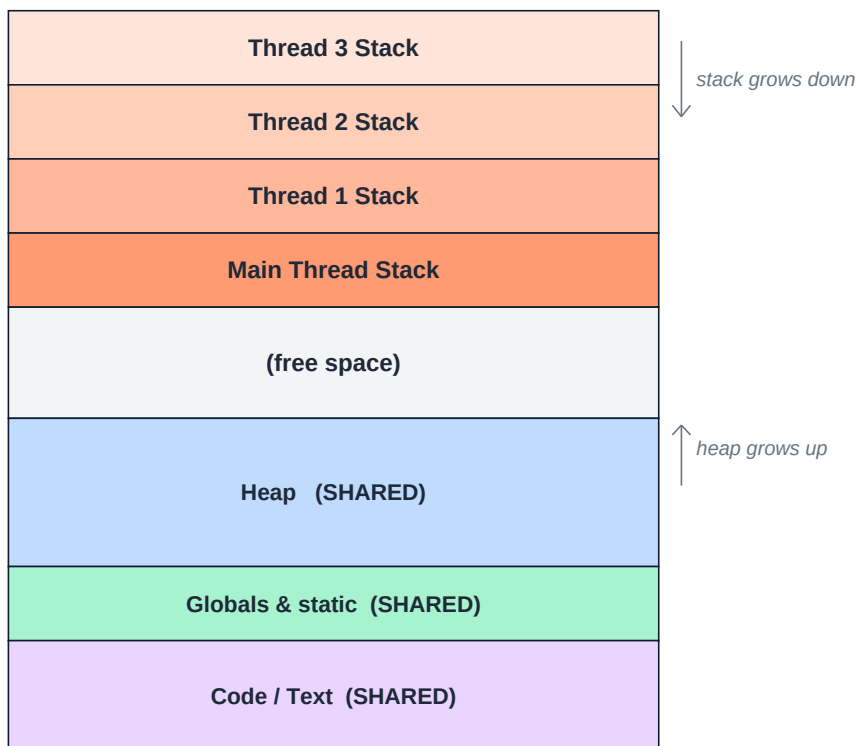
Heap: Dynamically allocated memory. Every `malloc()` carves out a piece of the heap. Every `free()` returns it. The heap grows upward.

Stack: Function calls happen here. Every time a function is called, a new stack frame is pushed containing the function's local variables, its return address, and where its arguments are. When the function returns, its frame is popped. The stack grows downward.

Now add threads

Here is the important picture. When you create additional threads inside a process, the code, globals, and heap stay exactly as they are — there's still only one of each, and all threads share them. But **each thread gets its own stack**. The OS allocates a fresh stack for every new thread.

high addresses



low addresses

With multiple threads, the stack region is split into one stack per thread. Heap, data, and code are still shared.

Memory region	Shared between threads?
Code (text)	Yes
Globals / static data	Yes
Heap (malloc'd memory)	Yes
Each thread's stack	No — each thread has its own

This is the single most important table in concurrency. Memorize it.

Why this matters

Look at this code:

```
int counter = 0;    // global — SHARED

void *worker(void *arg) {
    int local = 0;    // on this thread's stack — NOT SHARED
    for (int i = 0; i < 1000; i++) {
        local++;      // modifying a private variable, safe
        counter++;    // modifying a SHARED variable, dangerous
    }
    return NULL;
}
```

Two threads running `worker` simultaneously will each have their own `local` — no conflict. But they're both writing to the same `counter`. That is a problem, and the rest of the week (and most of the next) is about why.

Same thing with the heap. If two threads both have a pointer to the same `malloc'd` buffer, they're looking at the same memory. Modifications by one are immediately visible to the other.

A subtle point: pointers across stacks

A thread's stack is private to that thread, *but* if the thread passes a pointer to one of its stack variables to another thread, the other thread can technically follow that pointer and read or write the memory. It is "private" in the sense that no one else *should* touch it, not in the sense that it's physically inaccessible.

```
void *worker(void *arg) {
    int local = 10;
    pthread_create(..., another_thread, &local);    // bad idea
    return NULL;
    // 'local' is GONE after this, but another_thread
    // might still be using its address — crash time.
}
```

The general rule is: if you want a thread to share data with another thread, put the data somewhere shared (the heap, or a global) rather than in a thread's stack.

SECTION 03

Race Conditions: The Mayhem Begins

*This is the most important section of the week. The single concept here — the **race condition** — is responsible for a huge fraction of all bugs in concurrent software, including bugs in commercial operating systems, databases, and web browsers.*

The setup

Consider a piece of code any C programmer would call trivial:

```
counter++;
```

One line. One operation. What could go wrong?

Now imagine two threads both running this line on a shared counter. Suppose `counter` starts at 0 and each thread runs `counter++` once. What do you expect the value of `counter` to be afterward?

Two? Obviously two?

Sometimes. Sometimes one. This is a race condition.

What "counter++" actually is

The trick is that `counter++` is not one operation, even though it looks like one in C. The CPU doesn't have an instruction that says "go increment the variable named `counter` in one atomic step." It has instructions that do small, simple things. The compiler turns `counter++` into something like three machine instructions:

```
1. load  counter into register R
2. add   1 to R
3. store R back into counter
```

Now imagine two threads, A and B, both executing this sequence. The hardware can — and on a multi-core machine, will — interleave their instructions in any order. Here is one perfectly legal interleaving (counter starts at 0):

Time	Thread A	Thread B	counter
1	load counter into R_A		0
2		load counter into R_B	0
3	add 1 to R_A (R_A = 1)		0
4		add 1 to R_B (R_B = 1)	0
5	store R_A into counter		1
6		store R_B into counter	1

Both threads loaded the same starting value (0). Both computed 1. Both wrote 1 back. The two increments effectively merged into one. `counter` is now 1, not 2.

There is no bug in the C code in the usual sense. The code says to increment a counter. The problem is that incrementing a shared counter is not a single operation, and the threads' instructions can be interleaved in a way that makes them step on each other.

Why does this interleaving happen?

Two reasons.

On a single CPU core, the operating system can pause your thread between any two machine instructions and switch to another thread. This is called a **context switch**, and it's exactly how a single-core machine appears to "run many programs at once." Even on a single core, your `load/add/store` sequence can be interrupted right in the middle.

On multiple CPU cores, two threads are literally executing instructions at the same time, on different physical cores, each with their own registers. They might both `load counter` in the same nanosecond. The hardware doesn't coordinate them automatically.

Critical sections

A **critical section** is a piece of code that accesses shared data and that — for correctness — should be treated as a single, indivisible unit. In our example, the three instructions making up `counter++` form a critical section.

***The general rule of concurrency:** If two threads execute the same critical section concurrently on the same shared data, you have a race condition.*

Race conditions in disguise

The counter example is the simplest case. But race conditions don't always look like `counter++`. They hide in much more innocent-looking code.

Check-then-act

```
if (account_balance >= amount) {    // check
    account_balance -= amount;      // act
}
```

This is exactly the bug behind "how did the bank let me overdraw my account?" stories. The check and the act are two separate operations. Between them, another thread can also pass the check, and both threads can withdraw — and the balance can go negative.

Lazy initialization

```
if (config == NULL) {
    config = load_config_from_disk();
}
```

Two threads can both see `config == NULL`, both load it, and you end up with two `config` objects (one leaked) — or worse, depending on the loader.

Linked list insertion

```
new_node->next = head;
head = new_node;
```

Two threads inserting into a shared linked list with these two lines can lose nodes, create cycles, or produce a list that crashes when you try to traverse it.

The pattern is always the same: **multiple steps, shared data, no protection between the steps.**

How to recognize one

Two questions to ask of any piece of code:

1. Is this data shared between threads? (Globals, heap data accessible via shared pointers, anything not on a single thread's stack.)
2. Does my logic depend on a sequence of operations on this data happening as one unit? (Read-then-write, check-then-act, increment, modifying multiple fields together.)

If both answers are yes, you have a potential race condition. If you can't tell, assume yes. Concurrency bugs are infinitely more painful than the small overhead of a lock.

The terrifying thing

Race conditions are *non-deterministic*. The same program with the same input can produce different output every time you run it.

Worse, they are usually *rare*. The "bad" interleaving might happen 1 time in 1,000, or 1 in a million. Your buggy program will appear to work correctly for weeks, pass every test, get deployed to production, and then start corrupting your customers' data on a Tuesday at 2 AM.

This is why concurrent code can't be debugged by the usual "run it and see if it works" approach. You need to actively reason about *all possible interleavings* and either convince yourself none of them are bad, or add synchronization that prevents the bad ones. Week 2 is about the second option.

Thought experiment

Two threads. Both run this code:

```
if (x == 0) {  
    x = 1;  
} else {  
    x = 2;  
}
```

x starts at 0. What are the possible final values of x after both threads finish?

Answer: 1 or 2. Both threads can see `x == 0`, both can write 1. Or one writes 1, then the other sees `x == 1` and writes 2. Or both write 1 simultaneously. There's no way to get a value that didn't come from somewhere, but you can't predict which one you get.

SECTION 04

Pthreads Basics

Time for the actual API. POSIX threads (pthreads) is the standard C interface for creating and managing threads on Unix-like systems. It is what Linux, macOS, and the BSDs provide.

The header and the linker flag

Every pthreads program needs:

```
#include <pthread.h>
```

And when you compile it:

```
$ gcc -Wall -pthread my_program.c -o my_program
```

The `-pthread` flag is important. It links your program against the pthreads library and sets some compiler flags so pthread-related macros work correctly. If you forget it, you'll get a linker error like `undefined reference to pthread_create`. That's the signal.

Creating a thread: `pthread_create`

```
int pthread_create(
    pthread_t *thread,           // OUT: handle for new thread
    const pthread_attr_t *attr,  // attributes; pass NULL for default
    void *(*start_routine)(void *), // function the thread will run
    void *arg                    // argument passed to that function
);
```

thread — pointer to a `pthread_t`, filled in by the call with a handle you can use to refer to the thread later.

attr — thread attributes like stack size. Pass `NULL` for defaults; that's what you'll want this week.

start_routine — the function the new thread will start executing. The signature is fixed: it takes a `void *` and returns a `void *`.

arg — the argument passed to your `start_routine`.

Return value — 0 on success, error code on failure. Unlike most Unix calls, pthread functions return the error code directly rather than setting `errno`.

Why void *?

C doesn't have generics. The pthreads library needs to be able to run any function as a thread, regardless of what type its argument is and what type it returns. The standard C idiom for "a pointer to anything" is `void *`. Cast as needed:

```
int id = 42;
pthread_create(&t, NULL, worker, &id);

void *worker(void *arg) {
    int my_id = *(int *)arg;
    // ...
}
```

A common pitfall

People sometimes pass the loop variable directly:

```
for (int i = 0; i < 5; i++) {
    pthread_create(&threads[i], NULL, worker, &i); // BUG
}
```

All five threads end up with a pointer to the same `i`, which will be whatever value the loop variable has when each thread reads it. The fix is to give each thread its own value:

```
int ids[5];
for (int i = 0; i < 5; i++) {
    ids[i] = i;
    pthread_create(&threads[i], NULL, worker, &ids[i]);
}
```

Waiting for a thread: pthread_join

```
int pthread_join(pthread_t thread, void **retval);
```

`pthread_join` blocks the calling thread until the specified thread has finished. If `retval` is not `NULL`, the thread's return value is stored there.

Join every thread you create. If main returns while other threads are still running, the whole process is torn down — those threads die mid-execution, which is rarely what you want.

Other handy calls

`pthread_t pthread_self(void)` — returns the calling thread's own handle. Useful when a thread wants to print its own identity for debugging.

`int pthread_equal(pthread_t a, pthread_t b)` — compares two thread handles. Don't compare them with `==`; the type is opaque.

`void pthread_exit(void *retval)` — ends the calling thread early. Returning from the start routine has the same effect — use `pthread_exit` only when you want to exit from deep inside the call stack.

A complete template

Here is the canonical structure of every simple multithreaded program in this project:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4

void *worker(void *arg) {
    int my_id = *(int *)arg;
    printf("Hello from thread %d\n", my_id);
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i;
        if (pthread_create(&threads[i], NULL, worker, &ids[i]) != 0) {
            perror("pthread_create");
            exit(1);
        }
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("All threads done.\n");
    return 0;
}
```

Compile and run:

```
$ gcc -Wall -pthread template.c -o template
$ ./template
```

You'll see four "Hello from thread N" lines. **Run it several times.** The order changes between runs. That ordering is not guaranteed anywhere — the OS chooses when each thread gets to run, and the order can differ run to run. This is your first taste of concurrent non-determinism. Get used to it.

You now have everything you need to write multithreaded programs and to recognize how they go wrong. Head to the code examples in `code/`, run each one several times, and then tackle the problems in `problem-statement.pdf`.